

SPM Modulo 1: Vettorizzazione del kernel di Partition Mapping

(Versione italiana — solo per uso personale, non inclusa nella consegna)

Gabriele Marsili

1 Scelte progettuali

Dati N chiavi (`uint64_t`) e P partizioni (potenza di due), il compito è calcolare $\text{part_id}[i] = h(\text{keys}[i]) \in [0, P)$ per ogni chiave.

1.1 Funzione di mapping

Ho scelto una hash Fibonacci multiply-shift che opera su 32 bit:

$$h(k) = ((k_{\text{lo}} \oplus k_{\text{hi}}) \cdot A_{32}) \gg (32 - \log_2 P)$$

dove $A_{32} = 0x9E3779B9 = \lfloor 2^{32}/\varphi \rfloor$ [1]. Si tratta di un'istanza dello schema di *hashing by multiplication* descritto in [2] (§8.2), dove la costante legata al rapporto aureo $A = (\sqrt{5} - 1)/2 \approx 0.618$ è raccomandata per le sue proprietà di dispersione dei bit. L'estrazione tramite right-shift segue la famiglia universale di hash *multiply-add-shift* di [2] (§8.3): $h_{a,b}(k) = (ak + b \bmod 2^w) \div 2^{w-\ell}$, qui semplificata con $b=0$ e la costante di Fibonacci come a .

Lo XOR delle due metà a 32 bit preserva l'informazione di entrambe le metà prima della moltiplicazione. La costante di Fibonacci distribuisce bene i bit nell'intervallo di output: la distribuzione misurata su 10^8 chiavi con $P=256$ dà $\max(\text{count})/\text{atteso} = 1,005$.

La ragione principale per lavorare a 32 bit è la compatibilità SIMD: `_mm256_mullo_epi32` (`vpmulld`) è una singola istruzione nativa AVX2 che moltiplica 8 valori `uint32` in parallelo. Una moltiplicazione a 64 bit in AVX2 richiederebbe 3 istruzioni `vpmuludq` più shift e addizioni per elemento — ho verificato che risulta più lenta della baseline scalare in questo regime limitato dalla banda di memoria.

1.2 Layout dei dati e P

Chiavi e partition ID di output risiedono in array contigui separati con stride unitario — il layout più favorevole alla vettorizzazione. La memoria è allineata a 32 byte tramite `posix_memalign` per i load allineati AVX2. P è una potenza di due, così il modulo si riduce a un right-shift.

2 Auto-vettorizzazione

Il kernel C++ “plain” viene compilato da un singolo file sorgente in due binari: uno con `-fno-tree-vectorize` (baseline), l'altro con `-O3 -march=native -mavx2 -ftree-vectorize` (auto-vec). Entrambi usano `-fopt-info-vec-optimized` e `-fopt-info-vec-missed` per la diagnostica.

GCC 12.2 riporta la vettorizzazione riuscita del loop del kernel. Le righe rilevanti dal report completo (file `vec_report_optimized.txt`, incluso nei deliverable):

```
src/plain.cpp:33:26: optimized: loop vectorized
    using 32 byte vectors
src/plain.cpp:33:26: optimized: loop vectorized
    using 16 byte vectors
include/common.hpp:77:27: optimized: loop vectorized
    using 32 byte vectors
include/common.hpp:264:19: optimized: loop vectorized
    using 32 byte vectors
```

GCC emette due versioni del loop del kernel: una che usa registri AVX2 a 256 bit (`ymm`, vettori da 32 byte, 8 `uint32` per istruzione) e una che usa registri SSE a 128 bit (`xmm`, vettori da 16 byte, 4 `uint32` per istruzione). A runtime viene preso il percorso a 256 bit quando AVX2 è disponibile; la versione a 128 bit serve come fallback su hardware più vecchio. Le righe relative a `common.hpp` corrispondono ai loop di checksum e generazione chiavi, che GCC vettorizza anch'essi. Il report `-fopt-info-vec-missed` è risultato vuoto — GCC non ha trovato alcun loop che abbia tentato di vettorizzare senza riuscirci.

Il loop soddisfa tutte le condizioni: trip count noto, nessuna dipendenza loop-carried, nessuna chiamata a funzione nel corpo, `__restrict__` per escludere l'aliasing, e accesso con stride unitario.

3 Intrinsics AVX2

La versione scritta a mano processa 8 chiavi per iterazione. Due load `ymm` caricano 4+4 chiavi `uint64`. Ogni chiave viene XOR-folded shiftando a destra di 32 bit e facendo XOR con il valore originale, lasciando il risultato folded nei 32 bit bassi di ogni lane a 64 bit. Gli 8 valori folded vengono poi impacchettati in un singolo registro `ymm`: `permutevar8x32` con indici `[0,2,4,6,1,3,5,7]` estrae i valori low-32 da ciascun registro, e `permute2x128` con selettore `0x20` combina i 128 bit inferiori di entrambi in un unico registro di $8 \times \text{uint32}$. Un singolo `vpmulld` esegue tutte le 8 moltiplicazioni Fibonacci, un right-shift estrae i partition ID, e uno store a 256 bit scrive i risultati. Le restanti $N \bmod 8$ chiavi sono gestite da una coda scalare.

Il binario AVX2 verifica la correttezza rispetto a un riferimento scalare interno (compilato con `-fno-tree-vectorize` tramite `pragma`) confrontando i checksum FNV-1a prima del benchmarking; per $N \leq 32$ stampa anche il confronto elemento per elemento.

4 Analisi delle prestazioni

Tutti i benchmark sono stati eseguiti su node09 (AMD EPYC 7301, Zen 1, DDR4-2666) con allocazione SLURM esclusiva (`-partition=gpu-excl`), 11 ripetizioni, mediana riportata.

4.1 Risultati

La Tabella 1 riporta throughput e speedup su cinque dimensioni dell'input ($P=256$).

N	Baseline	Autovec		AVX2	
	Mkeys/s	Mkeys/s	S	Mkeys/s	S
10^6	929	1377	1,48×	1276	1,37×
10^7	915	1274	1,39×	1172	1,28×
5×10^7	908	1314	1,45×	1168	1,29×
10^8	918	1310	1,43×	1200	1,31×
2×10^8	908	1322	1,46×	1152	1,27×

Tabella 1: Throughput e speedup CPU rispetto alla baseline con $P=256$.

Con $N = 10^8$ il kernel CUDA da solo raggiunge 67 320 Mkeys/s (808 GB/s, 81% del picco HBM2 dell'A30), ma end-to-end inclusi i trasferimenti PCIe solo 1 031 Mkeys/s (1,12×); vedere Sezione 5.

4.2 Perché lo speedup è limitato

Ogni chiave richiede 12 byte di traffico di memoria (8 B lettura + 4 B scrittura) e solo ~ 4 operazioni intere. L'intensità operativa è $I = 4/12 \approx 0,33$ ops/byte, collocando il kernel nella regione bandwidth-bound del modello roofline. La banda single-core su node09 è stata misurata a $\sim 16,4$ GB/s; la versione autovec raggiunge 15,7 GB/s, circa il 96% di questo tetto (Figura 1). SIMD accelera la parte di calcolo, ma i trasferimenti di memoria — che dominano — restano invariati; lo speedup di 1,43× deriva da un utilizzo più efficiente della banda, non da un calcolo più veloce.

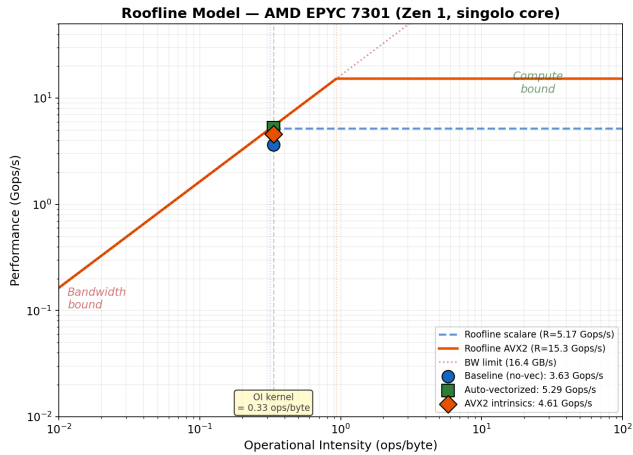


Figura 1: Modello roofline. Tutte le varianti si trovano sulla rampa di banda ($I=0,33$), ben al di sotto dei tetti di calcolo.

4.3 Autovec vs. intrinsics AVX2

In modo forse controintuitivo, la versione vettorizzata dal compilatore supera gli intrinsics scritti a mano (15,7 vs 14,4 GB/s). Poiché il kernel è limitato dalla memoria, le differenze marginali nello scheduling delle istruzioni, loop unrolling e allocazione dei registri che GCC applica durante l'auto-vettorizzazione si traducono in un utilizzo della banda leggermente migliore. Il codice con intrinsics impone una sequenza di istruzioni rigida che il compilatore non può facilmente riordinare.

4.4 Sweep dei parametri

N-sweep (Figura 3): lo speedup è stabile su tutte le dimensioni dell'input, da 10^6 a 2×10^8 chiavi. Questo conferma che il kernel è bandwidth-bound indipendentemente da N .

P-sweep (Figura 2a): throughput e speedup sono indipendenti da P (tutte le curve piatte su $P \in [2, 1024]$), confermando che cambiare P modifica solo la costante di shift — traffico di memoria e calcolo per chiave restano invariati.

Key-space sweep (Figura 2b): il throughput è invariante rispetto alla dimensione del key-space (tasso di duplicati), come atteso per una hash branchless a costo costante per chiave.

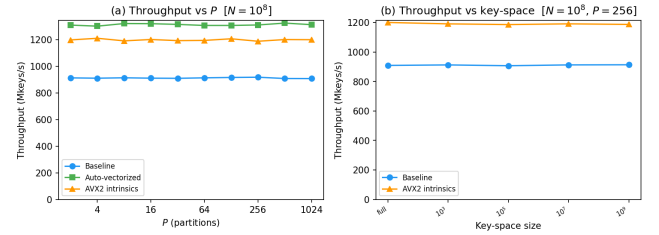


Figura 2: (a) Throughput vs. P con $N = 10^8$: piatto su tutti i valori di partizioni. (b) Throughput vs. dimensione del key-space con $N = 10^8$, $P = 256$: invariante al tasso di duplicati.

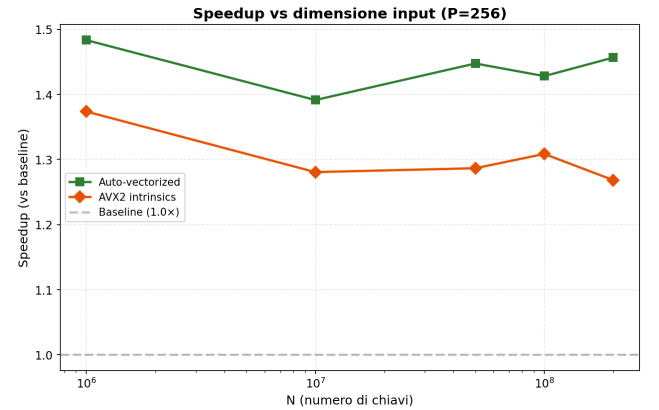


Figura 3: Speedup vs. dimensione dell'input N ($P=256$). Sia auto-vectorized che AVX2 intrinsics mantengono uno speedup stabile su tutti gli N , coerente con un kernel bandwidth-bound.

4.5 Stabilità delle misure

Tutti i 55 punti di misura hanno coefficiente di variazione inferiore al 5%. Il binario autovec è il più stabile (CoV medio 0,4%), mentre gli intrinsics AVX2 mostrano varianza leggermente maggiore (CoV medio 3,5%), probabilmente perché lo scheduling di GCC produce pattern di accesso alla memoria più deterministici della sequenza scritta a mano.

5 CUDA

Il kernel GPU usa la stessa hash, un thread per chiave, 256 thread/blocco. La correttezza è verificata tramite confronto di checksum FNV-1a rispetto al riferimento CPU; per $N \leq 32$ viene stampato anche il confronto elemento per elemento.

Fase	Tempo (ms)	BW (GB/s)	%
Trasferimento H→D	63,2	12,7	65%
Kernel	1,5	808	2%
Trasferimento D→H	32,3	12,4	33%
End-to-end	97,0	—	100%

Tabella 2: Tempi CUDA con $N = 10^8$, $P = 256$.

Il kernel da solo raggiunge 808 GB/s (81% del piccolo HBM2 dell'A30, pari a 993 GB/s). I trasferimenti PCIe pesano per il 98% del tempo end-to-end, quindi scaricare questo kernel isolato sulla GPU non dà vantaggi rispetto alla CPU. La GPU converrebbe solo con dati già presenti sul device o con stadi successivi della pipeline anch'essi su GPU.

6 Conclusioni

Questo kernel è un classico workload memory-bound. Con $I = 0,33$ ops/byte, le prestazioni sono dettate dalla banda DRAM indipendentemente dalla strategia SIMD. La versione autovec raggiunge il 96% del tetto di banda single-core, lasciando praticamente nessun margine di miglioramento su singolo thread. La scelta di una hash a 32 bit (vpmulld nativo) rispetto a una a 64 bit è stata fondamentale: con una hash a 64 bit, AVX2 era in realtà *più lento* dello scalare a causa dell'overhead della decomposizione vpmuludq in un regime limitato dalla banda.

Riferimenti bibliografici

- [1] D.E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998, § 6.4, pp. 513–558.
- [2] P. Ferragina, *Pearls of Algorithm Engineering*. Cambridge University Press, 2023, Cap. 8: The Dictionary Problem, pp. 96–127.